

W3C XML Schema Definition Language

- En l'especificació XML es fa referència a les DTD com a mètode per definir vocabularis XML
- les DTD tenen una sèrie de limitacions que van portar el W3C a definir una nova especificació.
- **W3C XML Schema Definition Language** (que popularment s'anomena **XML Schema** o **XSD**)
- es va crear per substituir la DTD com a mètode de definició de vocabularis per a documents XML.
- www.w3.org/XML/Schema.

Relax NG

Durant l'especificació molts membres del grup de treball van considerar que el que sortia era massa complex i van desenvolupar alternatives més simples, entre les quals destaca **Relax NG** (www.relaxng.org).

Com que **Relax NG** es va fer més tard que DTD i XSD, va poder solucionar alguns dels problemes.

L'èxit d'XSD ha estat molt gran i actualment es fa servir per a altres tasques a part de simplement validar XML. També es fa servir en altres tecnologies XML com XQuery, serveis web, etc.

Les **característiques** més importants que aporta XSD són:

- Està escrit en XML i, per tant, no cal aprendre un llenguatge nou per definir esquemes XML.
- Té el seu propi sistema de dades, de manera que es podrà comprovar el contingut dels elements.
- Suporta espais de noms per permetre barrejar diferents vocabularis.
- Permet ser reutilitzat i segueix els models de programació com herència d'objectes i substitució de tipus.

Associar un esquema a un fitxer XML

Per associar un document XML a un document d'esquema cal definir-hi l'espai de noms amb l'atribut **xmlns**, i per mitjà d'un dels atributs del llenguatge definir-hi quin és el fitxer d'esquema:

```
<lliga xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:noNamespaceSchemaLocation="lliga.xsd">
```

Definir referències a un esquema

Atribut	Significat
noNamespaceSchemaLocation	S'empra quan no es faran servir espais de noms en el document.
schemaLocation	S'empra quan es fan servir explícitament els noms dels espais de noms en les etiquetes.

Definir un fitxer d'esquema

L'XSD està basat en XML i, per tant, ha de complir amb les regles d'XML:

- Tot i que no és obligatori normalment sempre es comença el fitxer amb la declaració XML.
- Només hi ha un element arrel, que en aquest cas és <schema>.

Com que no s'està generant un document XML lliure sinó que s'està fent servir un vocabulari concret i conegut per poder fer servir els elements XML sempre s'hi haurà d'especificar l'espai de noms d'XSD: "http://www.w3.org/2001/XMLSchema".

```
<?xml version="1.0" ?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
    . . .
</xs:schema>
```

En la declaració d'aquest espai de noms s'està definint que xs és l'àlies per fer referència a totes les etiquetes d'aquest espai de noms. Els àlies per a XSD normalment són xs o xsd, però en realitat no importa quin es faci servir sempre que es faci servir el mateix en tot el document.

L'etiqueta <schema> pot tenir diferents atributs, alguns dels quals podem veure:

Atributs de l'etiqueta "<schema>"

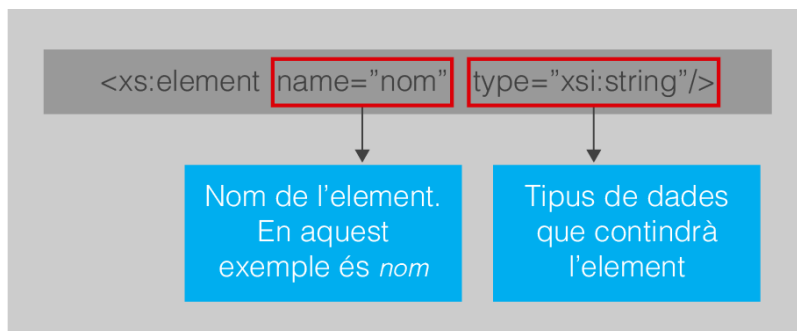
Atribut	Significat
attributeFormDefault	Pot ser qualified si fem que sigui obligatori posar l'espai de noms davant dels atributs o unqualified si no ho fem. Per defecte es fa servir unqualified.
elementFormDefault	Serveix per definir si cal l'espai de noms davant del nom. Pot prendre els valors qualified i unqualified.
version	Permet definir quina versió del document d'esquemes estem definint (no és la versió d'XML Schemas).

Etiquetes d'XSD

L'XSD defineix moltes etiquetes i no es veuran totes aquí. Podeu trobar totes les etiquetes possibles en l'especificació www.w3.org/TR/xmlschema11-1

Definició d'elements

Els elements es defineixen fent servir l'etiqueta <element>, i amb atributs s'hi especifica com a mínim el nom i opcionalment el tipus de dades que contindrà l'element.



L'XSD divideix els elements en dos grans grups basant-se en les dades que contenen:

- **Elements amb contingut de tipus simple:** són elements sense atributs que només contenen dades.
- **Elements amb contingut de tipus complex:** són elements que poden tenir atributs, no tenir contingut o contenir elements.

Elements amb contingut de tipus simple

Es consideren elements amb contingut de tipus simple aquells que no contenen altres elements ni tenen atributs.

Tipus	Dades que s'hi poden emmagatzemar
<i>string</i>	Cadenes de caràcters
<i>decimal</i>	Valors numèrics
<i>boolean</i>	Només pot contenir 'true' o 'false' o (1 o 0)
<i>date</i>	Dates en forma (AAAA-MM-DD)
<i>anyURI</i>	Referències a llocs (URL, camins de disc...)
<i>base64binary</i>	Dades binàries codificades en <i>base64</i>
<i>integer</i>	Nombres enters

La versió 1.1 d'XSD defineix una cinquantena de tipus de dades diferents, que es poden trobar a la seva definició (www.w3.org/TR/xmlschema11-2). Entre els més usats en destaquen els de la taula 2.9.

```
1 <xs:element name="posicio" type="xs:integer"/>
2 </xs:schema>
```

l'exemple següent no validarà:

```
1 <posicio>Primer</posicio>
```

Exemples:

Etiqueta	Exemple
<code><xs:element name="dia" type="xs:date" /></code>	<code><dia>2011-09-15</dia></code>
<code><xs:element name="alçada" type="xs:integer"/></code>	<code><alçada>220</alçada></code>
<code><xs:element name="nom" type="xs:string"/></code>	<code><nom>Pere Puig</nom></code>
<code><xs:element name="mida" type="xs:float"/></code>	<code><mida>1.7E2</mida></code>
<code><xs:element name="lloc" type="xs:anyURI"/></code>	<code><lloc>http://www.ioc.cat</lloc></code>

Quan es defineix una etiqueta en XSD s'està definint que l'etiqueta haurà de sortir en el document XML una vegada. És bastant habitual que hi hagi etiquetes que es repeteixin un nombre determinat de vegades. En XSD això s'ha simplificat per mitjà d'uns atributs de l'etiqueta <element> que determinen la **cardinalitat dels elements**:

- **minOccurs**: permet definir quantes vegades ha de sortir un element com a mínim. Un valor de '0' indica que l'element pot no sortir.
- **maxOccurs**: serveix per definir quantes vegades com a màxim pot sortir un element. unbounded implica que no hi ha límit en les vegades que pot sortir.

```
1 <xs:element name="nom" />
2 <xs:element name="cognom" maxOccurs="2"/>
```

També es poden donar valors als elements amb els atributs **fixed**, **default** i **nullable**.

L'atribut fixed permet definir un valor obligatori per a un element:

```
1 <xs:element name="centre" type="xs:string" fixed="IOC"/>

1 <centre />
2 <centre>IOC</centre>

1 <centre>Institut Cendrassos</centre>
```

A diferència de fixed, default assigna un valor per defecte però deixa que sigui canviat en el contingut de l'etiqueta.

```
1 <xsi:element name="centre" type="xs:string" default="IOC" />

1 <centre />
2 <centre>IOC</centre>
3 <centre>Institut Cendrassos</centre>
```

L'atribut nullable es fa servir per dir si es permeten continguts nuls. Per tant, només pot prendre els valors yes o no.

Tipus simples personals

Com que a vegades pot interessar definir valors per als elements que no han de coincidir necessàriament amb els estàndards, l'XSD permet definir tipus de dades personals. Per exemple, si es vol un valor numèric però que no accepti tots els valors sinó un subconjunt dels enters.

Per definir tipus simples personals no es posa el tipus a l'element i s'hi defineix un fill `<simpleType>`.

```
1 <xs:element name="persona">
2   <xs:simpleType>
3     ...
4   </xs:simpleType>
5 </xs:element>
```

Dins de `simpleType` s'especifica quina és la modificació que s'hi vol fer. El més corrent és que les **modificacions sigui fetes amb list, union, restriction o extension**.

Fer servir **list** permetrà definir que un element pot contenir llistes de valors. Per tant, per especificar que un element `<partits>` pot contenir una llista de dates es definiria:

```
1 <xs:element name="partits">
2   <xs:simpleType>
3     <xs:list itemType="xs:date"/>
4   </xs:simpleType>
5 </xs:element>
```

L'etiqueta validaria amb una cosa com:

```
1 <partits> 2011-01-07 2011-01-15 2011-01-21</partits>
```

Els elements `simpleType` també es poden definir amb un nom fora dels elements i posteriorment usar-los com a tipus de dades personal.

```
1 <xs:simpleType name="dies">
2   <xs:list itemType="xs:date"/>
3 </xs:simpleType>
4
5 <xs:element name="partits" type="dies"/>
```

Fent servir els tipus personalitzats amb nom es poden crear modificacions de tipus **union**. Els modificadors **union** serveixen per fer que es puguin mesclar tipus diferents en el contingut d'un element.

La definició de l'element <preu> farà que l'element pugui ser de tipus valor o de tipus símbol.

```
1 <xs:element name="preus">
2   <xs:simpleType>
3     <xs:union memberTypes="valor símbol"/>
4   </xs:simpleType>
5 </xs:element>
```

Amb això li podríem assignar valors com aquests:

```
1 <preus>25 €</preus>
```

Però sense cap mena de dubte el modificador més interessant és el que permet definir restriccions als tipus base. Amb el modificador **restriction** es poden crear tipus de dades en què només s'acceptin alguns valors, que les dades compleixin una determinada condició, etc.

L'element <naixement> només podrà tenir valors enters entre 1850 i 2011 si es defineix d'aquesta manera:

```
1 <xs:simpleType name="any_naixement">
2   <xs:restriction base="xs:integer">
3     <xs:maxInclusive value="2011"/>
4     <xs:minInclusive value="1850"/>
5   </xs:restriction>
6 </xs:simpleType>
7
8 <xs:element name="naixement" type="any_naixement"/>
```

Normalment els valors de les restriccions s'especifiquen en l'atribut value:

Atributs que permeten definir restriccions en XSD

Elements	Resultat
<code>maxInclusive</code> / <code>maxExclusive</code>	Es fa servir per definir el valor numèric màxim que pot agafar un element.
<code>minInclusive</code> / <code>minExclusive</code>	Permet definir el valor mínim del valor d'un element.
<code>length</code>	Amb <code>length</code> restringim la llargada que pot tenir un element de text. Podem fer servir <code><xs:minLength></code> i <code><xs:maxLength></code> per ser més precisos.
<code>enumeration</code>	Només permet que l'element tingui algun dels valors especificats en les diferents línies <code><enumeration></code> .
<code>totalDigits</code>	Permet definir el nombre de dígitos d'un valor numèric.
<code>fractionDigits</code>	Serveix per especificar el nombre de decimals que pot tenir un valor numèric.
<code>pattern</code>	Permet definir una expressió regular a la qual el valor de l'element s'ha d'adaptar per poder ser vàlid.

Per exemple, el valor de l'element `<resposta>` només podrà tenir un dels tres valors “A”, “B” o “C” si es defineix d'aquesta manera:

```
1 <xs:element name="resposta">
2   <xs:simpleType>
3     <xs:enumeration value="A"/>
4     <xs:enumeration value="B"/>
5     <xs:enumeration value="C"/>
6   </xs:simpleType>
7 </xs:element>
```


Una de les restriccions més interessants són les **definides per l'atribut pattern**, que permet definir restriccions a partir d'expressions regulars. Com a norma general tenim que si **s'especifica un caràcter en el patró aquest caràcter ha de sortir en el contingut**; les altres possibilitats les podem veure a la taula

Definició d'expressions regulars

Símbol	Equivalència
.	Qualsevol caràcter
\d	Qualsevol dígit
\D	Qualsevol caràcter no dígit
\s	Caràcters no imprimibles: espais, tabuladors, salts de línia...
\S	Qualsevol caràcter imprimible
x*	El de davant de * ha de sortir 0 o més vegades
x+	El de davant de + ha de sortir 1 o més vegades
x?	El de davant de ? pot sortir o no
[abc]	Hi ha d'haver algun caràcter dels de dins
[0-9]	Hi ha d'haver un valor entre el dos especificats, amb aquests inclosos
x{5}	Hi ha d'haver 5 vegades el que hi hagi al davant dels claudàtors
x{5,}	Hi ha d'haver 5 o més vegades el de davant
x{5,8}	Hi ha d'haver entre 5 i 8 vegades el de davant

Fent servir aquest sistema es poden definir tipus de dades molt personalitzats. Per exemple, podem definir que una dada ha de tenir la forma d'un **DNI (8 xifres, un guió i una lletra majúscula)** amb aquesta **expressió**:

```
1 <xs:simpleType name="dni">
2   <xs:restriction base="xs:string">
3     <xs:pattern value="[0-9]{8}-[A-Z]" />
4   </xs:restriction>
5 </xs:simpleType>
```

A part de les restriccions també hi ha l'element **extension**, que serveix per afegir característiques extra als tipus. No té gaire sentit que surti en elements de tipus simple però es pot fer servir, per exemple, per afegir atributs als tipus simples (cosa que els converteix en **tipus complexos...**).